

B.TECH FINAL YEAR PROJECT

MediSync

AI-Powered Healthcare Middleware
The Complete Viva Guide

Every module, every endpoint, every important function — explained twice:
once in plain English, once in technical detail.

Use this document to prepare for examiner questions on architecture, AI integration
(Gemini, RAG, ChromaDB), data flow, business logic, and the design decisions behind
every screen.

Table of Contents

1.	What MediSync is — in plain English
2.	System architecture & tech stack
3.	Data model — MongoDB collections
4.	Authentication & role-based access
5.	Module A — Doctor (Smart Scribe)
6.	Module B — Hospital (Billing & Claim Submission)
7.	Module C — Insurer (Clearinghouse)
8.	Module D — Patient (Care Companion)
9.	Background services — LLM, RAG, OCR, Adjudication
10.	Glossary — terms you should be ready to define
11.	Likely viva questions & how to answer them

1. What MediSync is — in plain English

If you had to explain MediSync in one sentence: it's a middleware layer that sits between doctors, hospitals, insurers, and patients to automate insurance claim processing in Indian healthcare.

The problem we're solving

In India, **over 40% of health insurance claims get denied or delayed**. Why?

- Doctors prescribe drugs that aren't covered by the patient's policy — they don't check before writing the prescription.
- Hospital bills are confusing — patients can't tell what's covered and what isn't.
- Insurers review every claim manually — takes days, full of errors.
- Patients receive discharge summaries in English medical jargon they can't understand.
- Nobody checks on patients after discharge — complications go unnoticed.

How MediSync solves it

Four role-specific modules talk to one shared backend:

Module	For Whom	What it does
Smart Scribe	Doctor	Records consultation → AI structures it into a clinical note → checks each prescription against the patient's policy in real-time.
Hospital Admin	Hospital	Reviews the doctor's note, runs medication-coverage checks, builds the itemized claim, and submits to the insurer.
Clearinghouse	Insurer	Auto-adjudicates claims in seconds using a Python rule engine — approves standard cases, flags complex ones for human review.
Care Companion	Patient	Bill decoder, discharge translator + audio in 10 Indian languages, daily health check, claim tracker.

DESIGN THROUGH-LINE

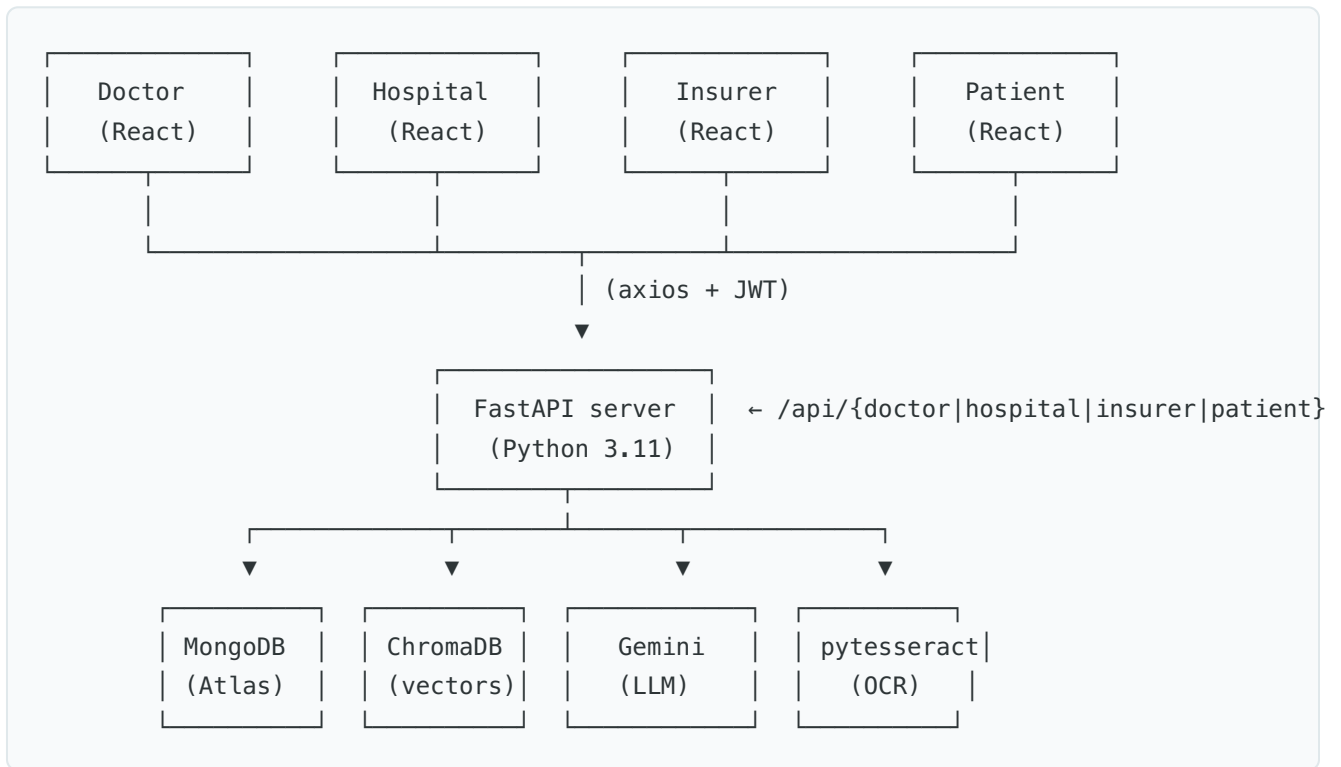
Every screen either **explains** (claims, bills, alerts in plain language) or **acts** (upload, submit, approve). No screen assumes medical literacy from the patient or programming literacy from anyone.

The Happy Path — one claim from start to finish

```
Patient uploads policy PDF
    ↓ (indexed once in ChromaDB)
Doctor records consultation
    ↓ (Web Speech API → transcript)
Smart Scribe → Gemini structures the transcript into JSON
    ↓ (symptoms, diagnosis, prescriptions, ICD codes)
RAG checks each prescription against the policy
    ↓ (warns about non-covered drugs)
Hospital reviews → may substitute drugs → builds itemized claim
    ↓
Insurer auto-adjudicates with Python rules
    ↓
Approved / Rejected / Flagged for manual review
    ↓
Patient sees status in plain language
```

2. System architecture & tech stack

The big picture



Tech stack — every choice has a reason

Layer	Choice	Why this and not the alternative
Frontend	React 19 + Vite	One repo for four dashboards via role-based routing. Vite gives sub-second hot reload.
Backend	FastAPI (Python)	Async, auto-generates OpenAPI/Swagger docs, Pydantic for type-safe request bodies.
Database	MongoDB Atlas (MO free)	Documents fit healthcare records (variable fields per claim). 512 MB free tier enough for demo.
LLM	Google Gemini (free tier)	Free API key from aistudio.google.com. HuggingFace + mock as fallbacks.
Vector DB	ChromaDB (local)	No cloud lock-in, no extra cost. Stores policy PDF embeddings on disk.
OCR	pytesseract	Offline, free Tesseract wrapper. Reads English hospital bills well.
Speech-to-text	Web Speech API	Built into Chrome — zero API cost, runs in the browser.
Text-to-speech	gTTS (Google TTS)	Free, supports Hindi, Bengali, Tamil, Telugu, etc.
Translation	deep-translator	Wraps Google Translate, free, covers 10+ Indian languages.
Auth	JWT + bcrypt	No third-party auth service needed. Token stored in localStorage on the client.
Charts	Recharts	React-native charting; pie, bar, line — all needed for the insurer analytics page.

LIKELY VIVA QUESTION

"Why didn't you use OpenAI or AWS?" — Both have free tiers that quickly hit limits and require credit cards. The brief was zero paid APIs. Gemini gives 60 req/min free, ChromaDB runs on the same disk, and Tesseract is fully offline. Every component is swappable via `.env`.

3. Data model — MongoDB collections

Eight collections cover every entity. All identified by Mongo's `_id` (ObjectId), surfaced to the frontend as a string `id`.

Collection	Stores	Key fields
<code>users</code>	Doctors, insurers, patients, hospitals (single collection, <code>role</code> field discriminates)	email, hashed_password, role, full_name, policy_number (patient only), license_number (doctor only)
<code>clinical_notes</code>	Output of Smart Scribe — structured consultation	doctor_id, patient_id, transcript, structured (symptoms/diagnosis/prescriptions/ICD), fhir, policy_warnings
<code>claims</code>	Insurance claims, regardless of who submitted	doctor_id, patient_id, hospital_id, items[], total_amount, status (pending → adjudicated → approved/rejected), flag_reasons, recommended_amount
<code>health_checks</code>	Post-discharge daily check-ins from patients	patient_id, doctor_id, wound_condition, fever, temperature, pain_level, is_flagged, flag_reasons
<code>policy_documents</code>	Pointer to indexed policy PDFs	policy_id, patient_id, insurer_name, file_path, chunks_indexed
<code>bill_analyses</code>	OCR + LLM analysis of patient-uploaded bills	patient_id, raw_text, ocr_items, llm_analysis
<code>daily_bills</code>	Hospital's per-day itemized billing	hospital_id, patient_id, items[], insurer_items[], patient_items[], insurer_total, patient_total, patient_paid
<code>discharge_translations</code>	Discharge summaries simplified + translated + audio path	patient_id, original_text, simplified_text, language, translated_text, audio_path

Why MongoDB and not Postgres?

Healthcare records are **inherently variable** — one claim has a room rent line, the next has 14 medications, the third has just an OPD consultation. Document store > rigid schema. Plus FastAPI + Motor gives async Mongo access out of the box.

4. Authentication & role-based access

Login flow (non-technical)

User types email + password. Server checks the password hash. If valid, server returns a **JWT token** — a signed string the client stores. Every subsequent request attaches this token in the `Authorization` header so the server knows who's calling.

The code path

Frontend entry

`client/src/context/AuthContext.jsx` — `login()` + `register()` functions.

API client

`client/src/utils/api.js` — axios instance with request interceptor that auto-attaches the JWT.

Backend route

`POST /api/auth/login` in `server/routes/auth.py`.

Token creation

`create_access_token()` in `server/middleware/auth_middleware.py`.

Role guard

`require_role(["doctor"])` dependency in FastAPI routes.

Why JWT and not session cookies?

- JWTs are **stateless** — server doesn't need to remember sessions, scales horizontally.
- Self-contained — payload includes `user_id` + role, so role checks don't require a DB hit.
- bcrypt for password hashing — same algorithm used by Auth0, Django, GitHub. Salt + slow hash = brute-force resistant.

Role-based UI

Login redirects to `/doctor`, `/hospital`, `/insurer`, or `/patient` based on `user.role`. The sidebar (`Layout.jsx`) renders a different menu per role from a single `MENU_BY_ROLE` object.

`ProtectedRoute.jsx` blocks unauthorized paths.

5. Module A — Doctor (Smart Scribe)

The doctor talks. The AI does the rest.

Pages in this module

Page	Path	Function
Doctor Dashboard	/doctor	Quick KPIs + nav to other pages.
Consultation	/doctor/consultation	The Smart Scribe page — speech-to-text, AI structuring, real-time policy check.
Clinical Notes	/doctor/notes	Historical record of every consultation, with expandable details (symptoms, ICD codes, FHIR, safety flags).
Patient Alerts	/doctor/alerts	Inbox of post-discharge health checks flagged as concerning.
Upload Policy	/doctor/upload-policy	Optional — most policies are uploaded by the patient or hospital.

The Smart Scribe pipeline — step by step

1. **Record** — doctor clicks ; browser's Web Speech API converts audio → transcript. Zero API cost.
2. **Process with AI** — transcript sent to `POST /api/doctor/structure-note`.
3. **LLM structuring** — `structure_clinical_note(transcript)` in `llm_service.py` sends Gemini a strict prompt that mandates: symptoms array, primary diagnosis (not rule-outs), prescriptions with dosage/frequency/duration/ `is_new` / `stopped`, ICD-10 codes, recommended tests, safety flags.
4. **Post-processing** — `_post_process_structured_note()` uses regex on the original transcript to catch monitoring instructions Gemini sometimes skips (e.g. "blood sugar 4 times daily").
5. **RAG coverage check** — for each prescription, `check_medication_coverage()` in `rag_service.py` queries ChromaDB on the patient's policy, sends the relevant chunks to Gemini, gets back `{is_covered, alternative}`.
6. **FHIR resource** — `_build_fhir_encounter()` in `doctor.py` wraps everything in a FHIR-compliant Encounter JSON — the global standard for healthcare data interoperability.
7. **Persist** — saved to `clinical_notes` collection with `policy_warnings[]` if any meds weren't covered.

Important functions in this module

Function	File	What it does
<code>structure_clinical_note()</code>	<code>llm_service.py</code>	The Gemini call that turns transcript → JSON. Sends a long prompt with extraction rules.
<code>_post_process_structured_note()</code>	<code>llm_service.py</code>	Regex-based safety net that re-extracts tests / safety flags from the raw transcript if the LLM missed them.
<code>check_medication_coverage()</code>	<code>rag_service.py</code>	RAG check: pulls policy chunks, asks LLM coverage question per drug.
<code>_build_fhir_encounter()</code>	<code>doctor.py</code>	Builds a FHIR Encounter resource — interoperable healthcare format.
<code>_evaluate_health_check()</code>	<code>patient.py</code>	Rule engine for daily check; flagged items show up in Patient Alerts.

Patient Alerts — how the rule engine works

This is a **deterministic rule engine**, not an LLM. Every condition is explicit:

Condition	Flag type
Wound is bleeding or has discharge	Severe
Wound is red or swollen	Possible infection
Temperature ≥ 38.5 °C	Severe
Pain level ≥ 7 / 10	Severe
Appetite = none	Possible complication
Mobility = bedridden	Possible complication
Medication not taken	Adherence concern

`is_flagged = (len(flags)  $\geq$  2) OR (any severe flag)`. Flagged checks set `doctor_notified: true` so the doctor sees them in the Patient Alerts inbox.

6. Module B — Hospital (Billing & Claim Submission)

Where the doctor's note becomes money and paperwork.

Pages

Page	Path	Function
Hospital Dashboard	/hospital	KPIs — admitted patients, notes pending billing, claims submitted today.
Patient Records	/hospital/patients	Master list of patients with policy status + latest note.
Submit Claim	/hospital/submit-claim	Discharge-settlement flow: pick clinical note → coverage check → substitute drugs → submit one full claim.
Daily Bill	/hospital/daily-bill	Running tab during the stay; splits insurer vs patient per item.
Upload Policy	/hospital/upload-policy	Reception-side policy upload for walk-in patients.

The medication coverage check — RAG in action

Endpoint: `POST /api/hospital/check-medication-coverage` — `hospital.py` lines 189–321.

1. Look up the patient's policy in `policy_documents`.
2. `query_policy(policy_id, "drug formulary covered medications excluded ...")` — ChromaDB returns the policy chunks most semantically similar to that query.
3. Build a strict-pharmacist prompt for Gemini: *"For each of these medications, return covered=true/false, quote the exact policy section, and if excluded suggest the closest covered alternative from Section 3.2."*
4. Parse the JSON array, return per-drug status to the frontend.

WHY THIS MATTERS

Without this, the hospital submits a claim, the insurer rejects it three days later for "drug not in formulary", and the patient pays out of pocket. With this, the hospital sees coverage issues **before** the claim is submitted and can swap to a covered drug.

Two billing flows — Submit Claim vs Daily Bill

	Submit Claim	Daily Bill
When used	At discharge — one settlement.	During stay — line-by-line each day.
Input	A clinical note (doctor's prescriptions).	Manual entry, optional demo button.
Output	One claim → insurer.	Two buckets: covered items → insurer claim; uncovered items → patient pays at counter.
Coverage check	Per-medication (RAG).	Per-item (RAG, item by item).

Room rate constants

`ROOM_RATES` in `hospital.py` — the hospital's standard rates per bed type:

```
general:    ₹1,500/day
semi-private: ₹3,000/day
private:    ₹6,000/day
icu:        ₹12,000/day
```

The insurer rule engine has its own **caps** (different from the rates) — if the patient's policy is "standard" tier and the hospital charges ₹6,000 for a private room but the cap is ₹4,000, the rule engine flags **₹2,000 excess**. The claim still goes through but at reduced amount.

7. Module C — Insurer (Clearinghouse)

The brain of the operation. Where claims that took 3 days now take 60 seconds.

Pages

Page	Path	Function
Insurer Dashboard	/insurer	Count tiles + single button: "Adjudicate All Pending".
Claims	/insurer/claims	Tabbed worklist: Pending / Adjudicated / Flagged / Approved / Rejected. Single + batch actions. Locked once terminal.
Analytics	/insurer/analytics	Pie (by status), bar (top diagnoses), line (monthly trend) — Recharts.

The auto-adjudication rule engine

File: `server/services/adjudication_service.py`. **No LLM here** — insurance decisions must be auditable and reproducible. Pure Python rules.

#	Rule	What it checks	Effect
1	Policy tier	Policy number prefix (<code>PREM*</code> , <code>STD*</code> , <code>SUP*</code>) → tier.	Sets per-tier limits.
2	Max claim amount	Total amount vs tier ceiling (basic ₹3L, standard ₹5L, premium ₹10L, super ₹25L).	Flag + clamp.
3	Room rent cap	For private/deluxe/suite rooms — daily rate vs cap.	Flag + subtract excess.
4	Excluded items	Item descriptions containing cosmetic / dental / spectacles / vitamins / fertility / ivf.	Reject the line + deduct.
5	Manual-review ICD	Codes prefixed with <code>C</code> (cancer), <code>Z51</code> (chemo), <code>I21</code> (MI), <code>I63</code> (stroke), <code>K80</code> (gallstones), <code>N20</code> (kidney stones).	Force manual review.
6	Pre-auth keywords	Diagnosis mentions surgery / chemotherapy / dialysis / transplant / joint replacement / cardiac / angioplasty / bypass.	Force manual review.

Terminal statuses the engine produces

flagged

Complex case — needs human review. No recommendation attached. Triggered when manual review is required OR more than 2 flags fire.

adjudicated

Rules ran cleanly. The engine attached a recommendation: `approve` or `reject`, plus the recommended amount. **The human still presses the final button.**

KEY DESIGN DECISION — SEPARATION OF RECOMMENDATION AND DECISION

The engine never writes `approved` or `rejected` on its own. It writes `adjudicated` + a recommendation. The insurer presses the button. This creates an audit trail that regulators love: "machine recommended X, human did Y."

Important functions

Function	File	What it does
<code>adjudicate_claim(claim)</code>	<code>adjudication_service.py</code>	The rule engine. Returns status, recommendation, recommended_amount, flag_reasons.
<code>get_policy_tier()</code>	<code>adjudication_service.py</code>	Maps policy number prefix → tier (basic/standard/premium/super_premium).
<code>auto_adjudicate_claim()</code>	<code>insurer.py</code>	Runs the engine on one pending claim and persists the result.
<code>adjudicate_all_pending()</code>	<code>insurer.py</code>	Loops over up to 100 pending claims; returns counts.
<code>batch_approve_claims()</code>	<code>insurer.py</code>	Bulk-approve at recommended amount; silently skips non-adjudicated claims.
<code>get_analytics()</code>	<code>insurer.py</code>	Three Mongo aggregation pipelines: status, top diagnoses, monthly trend.

The "locked" state

Once a claim is `approved` or `rejected`, both endpoints (`/approve`, `/reject`) return 400 instead of mutating it. The frontend hides the action buttons and shows a locked summary with the final amount + date. **Decisions are final by design.**

8. Module D — Patient (Care Companion)

Every patient screen either explains or acts. Nothing assumes medical literacy.

Pages

Page	Path	Function
Patient Dashboard	/patient	Tiles + nav.
Upload Policy	/patient/upload-policy	One-time PDF upload — indexed in ChromaDB, reused by every downstream check.
My Claims	/patient/claims	Claim status with plain-language explanations.
Bill Decoder	/patient/bill-decoder	Upload hospital bill image → OCR + LLM analysis.
Discharge Summary	/patient/discharge	Paste English summary → AI simplifies → translates to vernacular → MP3 audio.
Health Check	/patient/health-check	Daily post-discharge form. Rule engine flags concerning answers.
Payable	/patient/payable	Uncovered daily-bill items the patient must pay at the counter.

The Bill Decoder pipeline

1. Patient uploads a JPEG/PNG/PDF of their hospital bill.
2. `extract_bill_items()` in `ocr_service.py` runs pyesseract on the image, then regex-parses lines like "Paracetamol 500mg ... 120.00" into `[{description, amount, category}]`.
3. `analyze_bill(items, policy_info)` in `llm_service.py` sends Gemini the line items + policy info and asks for per-item `{covered, explanation}`, plus total / covered_total / out_of_pocket / plain-English summary.
4. Frontend renders three cards (Total / Covered / Out-of-Pocket) and an item table with ✓/X icons.

The Discharge Translator pipeline

1. Patient pastes English discharge summary (full of jargon: "DVT prophylaxis", "PRN q6h", "antipyresis", etc.)
2. `simplify_discharge_summary()` — Gemini rewrites in plain English ("blood thinner to prevent clots").
3. `translate_and_speak()` in `translation_service.py` — `deep-translator` → target Indian language. Then `gTTS` generates MP3 saved under `audio_files/`.
4. Frontend shows simplified English, translated text, and a playable audio element with download.

The plain-language claim explainer

`_explain_claim_status()` in `patient.py` — a deterministic dict of templates, **not an LLM call**. Reads `status`, `total_amount`, `approved_amount`, `rejection_reason` and renders sentences like:

```
"Your claim for ₹12,185 needs additional review.  
Reason: Room rent ₹6,000/day exceeds cap ₹4,000/day."
```

Reason for not using an LLM: response must be instant and 100% predictable. Templates achieve both.

The Payable flow

When the hospital saves a daily bill, items that fail coverage land in `daily_bills.patient_items[]` with a positive `patient_total`. `GET /api/patient/payable` queries:

```
db.daily_bills.find({  
  patient_id: me,  
  patient_total: { $gt: 0 },  
  patient_paid: { $ne: true }  
})
```

`POST /api/patient/payable/{bill_id}/paid` sets `patient_paid: true` after collection at the counter.

9. Background services — LLM, RAG, OCR, Adjudication

LLM Service — `server/services/llm_service.py`

One module abstracts every LLM call. Provider chosen via `LLM_PROVIDER` env var: `gemini` (default), `huggingface`, or `mock` (returns hand-crafted JSON for offline demos).

Function	Used by	What it returns
<code>query_llm(prompt)</code>	Every other function	Plain text from the LLM. Retry + fallback chain (Gemini → HF → mock).
<code>structure_clinical_note(transcript)</code>	Doctor	JSON dict with symptoms, diagnosis, prescriptions, etc.
<code>analyze_bill(items, policy_info)</code>	Patient Bill Decoder	Per-item coverage + summary.
<code>simplify_discharge_summary(text)</code>	Patient Discharge	Plain English text.
<code>check_policy_coverage(item, policy_context)</code>	Hospital Daily Bill	<code>{is_covered, reason}</code> for one item.
<code>_extract_json(text)</code>	Helper	Pulls JSON out of LLM responses that may wrap it in markdown code blocks.

RAG Service — `server/services/rag_service.py`

RAG = Retrieval-Augmented Generation. The LLM doesn't know your specific policy PDF, so we **retrieve** the relevant chunks first, then **augment** the LLM prompt with them.

Function	What it does
<code>index_policy_pdf(path, policy_id, insurer)</code>	PyPDF2 reads the PDF → <code>chunk_text()</code> splits into ~500-word overlapping chunks → sentence-transformer embeds them → ChromaDB stores under <code>collection_name = policy_id</code> .
<code>query_policy(policy_id, query, n=3)</code>	Returns the top-3 most semantically similar chunks. This is the "retrieval" half of RAG.
<code>check_medication_coverage(drug, policy_id)</code>	Convenience wrapper — query the policy for that drug, send context to LLM, return covered/excluded + alternative.

WHY RAG AND NOT JUST ONE GIANT LLM CALL?

Policy PDFs are 30–60 pages. Sending all of that with every coverage check would (a) blow past context limits, (b) cost a fortune in tokens, (c) be slow. RAG sends only the 3 paragraphs that actually talk about the relevant drug.

OCR Service — `server/services/ocr_service.py`

pytesseract wrapper. `extract_bill_items()` opens the image with Pillow, runs Tesseract, then `_parse_bill_text()` regex-extracts line items. Categorization (medication, lab, room, etc.) via a keyword dictionary in `_categorize_item()`.

Adjudication Service — `server/services/adjudication_service.py`

Covered in Section 7. Zero AI. Pure Python rules. ~150 lines. Most-quoted file in viva.

Translation Service — `server/services/translation_service.py`

`translate_and_speak(text, lang)` — deep-translator + gTTS → MP3. `get_supported_languages()` returns the picker options: Hindi (hi), Bengali (bn), Tamil (ta), Telugu (te), Marathi (mr), Gujarati (gu), Kannada (kn), Malayalam (ml), Punjabi (pa), Urdu (ur), plus English.

Anonymization Service — `server/services/anonymization_service.py`

Privacy layer used when patient data is aggregated for analytics. Hashes direct identifiers (name, email), generalizes age to 5-year ranges, masks phone numbers, redacts Aadhaar / phone / email patterns from free text. K-anonymity style — patient still uniquely retrievable via ID, but features like name don't appear in analytics outputs.

10. Glossary — terms you should be ready to define

FHIR

Fast Healthcare Interoperability Resources. A global JSON-based standard from HL7 for sharing healthcare data. We output every clinical note in FHIR Encounter format so any other hospital system could ingest it without translation.

ICD-10

International Classification of Diseases, 10th revision. WHO's standard codes for every diagnosis. `J06.9` = acute upper respiratory infection. `I10` = essential hypertension. `C50.9` = breast cancer. The adjudication engine flags ICD prefixes like `C` for manual review.

RAG (Retrieval-Augmented Generation)

A pattern where you first retrieve relevant documents from a vector database, then feed those to an LLM as context. Solves the problem of LLMs not knowing private/proprietary data (like a specific insurance policy).

Vector embedding

A list of ~384–1536 floats representing the "meaning" of a piece of text. Texts with similar meanings have vectors with high cosine similarity. ChromaDB stores these so we can search by meaning, not just keyword.

ChromaDB

An open-source embedded vector database. Runs locally — no cloud account needed. We use one collection per policy PDF; chunks are embedded with sentence-transformers (the default).

JWT (JSON Web Token)

A signed token containing user identity. Stateless authentication — server doesn't need to track sessions. Composed of header.payload.signature; signature verifies the server's secret. We store user_id + role in the payload.

bcrypt

Adaptive password hashing function. Slow on purpose — defeats brute force. Includes a salt to defeat rainbow tables. Same algorithm used by Auth0, GitHub, Django.

Adjudication

The insurance term for "deciding whether a claim should be paid." In MediSync, adjudication is the rule-engine step. Result is one of `approved`, `rejected`, `flagged`, or `adjudicated` (recommendation only, awaiting human).

Pre-authorization

An insurance step where the patient must get insurer approval BEFORE a procedure (typical for surgery, chemo, transplant). The rule engine detects pre-auth-requiring keywords in the diagnosis and forces the claim into manual review.

Room rent cap

A clause in most Indian health policies — the policy will cover up to ₹X per day for room charges. If the patient picks a more expensive room, the excess comes out of pocket. The engine subtracts excess from the recommended amount.

Formulary

The list of drugs an insurance policy covers. RAG checks each prescription against the patient's specific formulary (Section 3.2 of most policies = covered drugs; Section 3.3 = exclusions).

OCR (Optical Character Recognition)

Turning an image of text into actual text characters. We use pytesseract (the Python wrapper for Google's open-source Tesseract engine). Free, offline, works well on printed English bills.

K-anonymity

A privacy principle: any individual should be indistinguishable from at least K others in the dataset. Our anonymization service generalizes age into ranges and hashes direct identifiers to support analytics queries without exposing individuals.

Motor

Async MongoDB driver for Python. Lets FastAPI's async functions await DB queries without blocking. Drop-in replacement for the sync PyMongo client.

Pydantic

Python library for data validation using type hints. Every request body in our API is a Pydantic model — automatic validation, automatic OpenAPI schema generation.

11. Likely viva questions & how to answer them

Q1: "WHY USE RAG INSTEAD OF FINE-TUNING?"

Fine-tuning needs labeled data, GPU compute, and re-training every time a policy changes. RAG works on day one with zero training — drop a new PDF in, index it, query it. Plus you can cite the exact policy section you used, which is critical for an auditable insurance decision.

Q2: "WHY IS THE ADJUDICATION ENGINE PURE PYTHON AND NOT AN LLM?"

Insurance decisions are **legally regulated** and must be 100% reproducible. LLMs are non-deterministic — same input can give different outputs across runs. Pure rules are inspectable: you can show the regulator the exact lines that fired. Also faster (~5 ms vs LLM's 2–3 s).

Q3: "WHAT HAPPENS IF GEMINI IS DOWN OR FAILS TO RETURN VALID JSON?"

The fallback chain in `llm_service.py`: Gemini → HuggingFace → mock. `_extract_json()` handles three response shapes (markdown code block, bare JSON, embedded). If all fail, the structured note returns a default with `diagnosis: "Requires manual review"` — the doctor sees this in the UI and re-records.

Q4: "HOW DO YOU HANDLE DRUG NAME TYPOS FROM SPEECH-TO-TEXT?"

The Smart Scribe prompt explicitly tells Gemini to **normalize drug names** (e.g. "livo citrzine" → "Levocetirizine") and to flag unverified drugs with `[UNVERIFIED DRUG]`. The doctor can review and correct before submitting.

Q5: "WHY IS THE DAILY BILL SPLIT PER-LINE AND NOT JUST A SINGLE PERCENTAGE?"

Real policies exclude **specific items** (cosmetic, dental, supplements) — not a flat percentage. So the patient owes ₹450 for the multivitamin and ₹800 for the cosmetic cream, NOT 30% of the total. Per-item attribution is what makes the patient's Payable page accurate.

Q6: "WHAT'S THE DIFFERENCE BETWEEN STATUS 'ADJUDICATED' AND 'APPROVED'?"

adjudicated = rule engine ran, recommendation attached, awaiting human button-press.
approved = insurer confirmed; final amount locked. Once approved, no further mutations allowed (backend returns 400; frontend hides buttons).

Q7: "HOW DOES THE SYSTEM KNOW WHICH DOCTOR TO ALERT WHEN A PATIENT SUBMITS A FLAGGED HEALTH CHECK?"

The health-check endpoint looks up the patient's **most recent clinical note** and uses its `doctor_id`. So whoever treated them last gets the alert. If no note exists, `doctor_id = "unassigned"`.

Q8: "CAN A PATIENT SEE ANOTHER PATIENT'S CLAIMS OR NOTES?"

No. Every patient route filters by `patient_id == current_user.id` from the JWT. The role guard `require_role(["patient"])` blocks anyone but a patient from even reaching these endpoints.

Q9: "WHY NOT STORE THE POLICY PDF ITSELF IN MONGODB?"

Two reasons: (1) MongoDB has a 16 MB document limit — fine for most policies but not all; (2) the indexed vectors in ChromaDB are what we actually query against, so the raw PDF is reference data only. We store the file path in `policy_documents` for re-indexing if needed.

Q10: "HOW DOES THE SYSTEM SCALE?"

FastAPI + Motor is async — one process handles ~5,000 concurrent requests. Stateless JWT means horizontal scaling needs no shared session store. MongoDB Atlas auto-scales. ChromaDB can be swapped for hosted Pinecone/Weaviate if needed. Bottleneck would be the Gemini free-tier (60 req/min) — solved by upgrading the API key or routing to the HF fallback.

Good luck with your viva. Every screen, every endpoint, every function explained — you've got this.